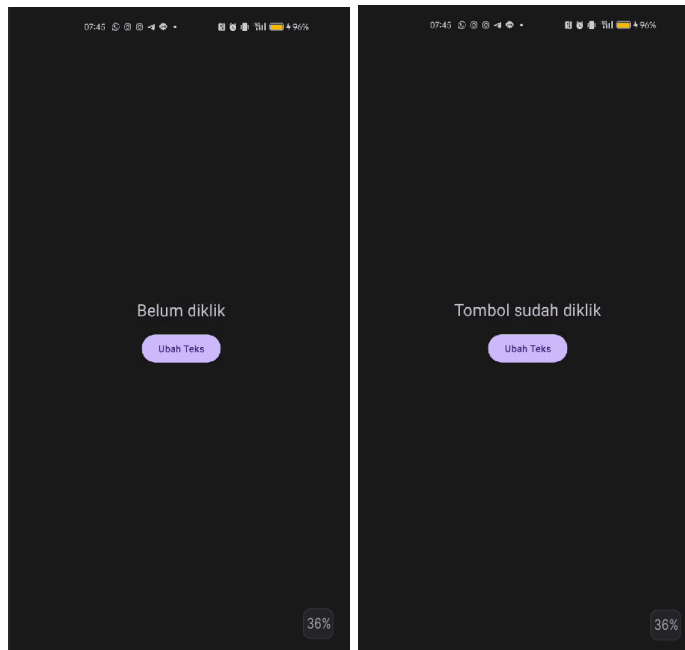


Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

Jawab:

Kondisi sebelum & setelah tombol di klik



2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Jawab:

Objek yang diubah secara langsung adalah objek TextView bernama tvMessage, tepatnya properti .text miliknya. Baris kunci: `tvMessage.text = "Tombol sudah diklik"`. Program mencari referensi View lewat `findViewById()`, lalu langsung memerintahkan objek tersebut mengganti isi propertinya. Baris kodenya:

```
val tvMessage = findViewById<TextView>(R.id.tvMessage)
```

```
val btnChange = findViewById<Button>(R.id.btnChange)
```

```
btnChange.setOnClickListener {  
    tvMessage.text = "Tombol sudah diklik"  
}
```

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Jawab:

Yang diubah bukan objek Text, melainkan nilai state bernama message. Baris kunci: message = "Tombol sudah diklik". Composable Text tidak pernah disentuh langsung; ia hanya "membaca" nilai message setiap kali digambar ulang. Baris kodenya:

```
var message by remember { mutableStateOf("Belum diklik") }  
Text(text = message, fontSize = 24.sp)  
Button(onClick = {  
    message = "Tombol sudah diklik"  
}) {  
    Text("Ubah Teks")  
}
```

4. Mengapa pada Compose tidak terdapat perintah seperti tvMessage.text atau setText()?

Jawab:

Karena Compose tidak mengenal konsep "mencari objek UI lalu mengubah propertinya" seperti View system. Di Compose, komponen Text bukan objek yang disimpan dan diberi perintah, melainkan hasil dari fungsi yang dipanggil ulang (recompose) setiap kali state berubah. Tampilan didefinisikan sebagai fungsi dari state (UI = f(state)), sehingga yang perlu dilakukan hanyalah mengubah sumber data (state), dan Compose sendiri yang akan menggambar ulang bagian UI yang bergantung pada state tersebut. Tidak ada referensi objek Text yang bisa "diperintah" seperti tvMessage.text karena arsitekturnya memang deklaratif, bukan imperatif.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Jawab:

- Event: pengguna menekan tombol btnChange, memicu setOnClickListener.
- View: di dalam listener, program mengambil referensi objek View (tvMessage) yang sudah didapat lewat findViewById().
- Perubahan tampilan: program langsung mengubah properti text pada objek tvMessage tersebut (tvMessage.text = "Tombol sudah diklik"), sehingga tampilan di layar langsung berubah karena objek View-nya sendiri yang dimodifikasi.

Jadi alurnya: klik tombol -> cari & pegang objek View -> ubah properti objek itu secara langsung -> layar berubah.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Jawab:

- Event: pengguna menekan tombol, memicu blok onClick.
- State: di dalam onClick, nilai variabel state message diubah (message = "Tombol sudah diklik"). Karena message dideklarasikan dengan mutableStateOf, Compose "mengamati" perubahan ini.
- Recomposition: Compose mendeteksi bahwa state yang diobservasi berubah, lalu secara otomatis memanggil ulang (recompose) fungsi composable yang bergantung pada message, dalam hal ini composable yang memanggil Text(text = message).
- Perubahan tampilan: saat recomposition terjadi, Text digambar ulang dengan nilai message yang baru, sehingga teks di layar berubah menjadi "Tombol sudah diklik".

Jadi alurnya: klik tombol -> ubah state -> Compose mendeteksi & memicu recomposition -> UI digambar ulang dengan nilai state terbaru.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: var message = "Belum diklik"? Jelaskan penyebabnya.

Jawab:

Jika message menjadi variabel biasa (var message = "Belum diklik"), teks di layar tidak akan berubah saat tombol diklik, karena Compose hanya memicu recomposition untuk state yang bertipe State<T> (hasil mutableStateOf). Variabel biasa tidak diobservasi oleh Compose, sehingga meski nilainya berubah di memori, UI tetap menampilkan nilai lama.

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Jawab:

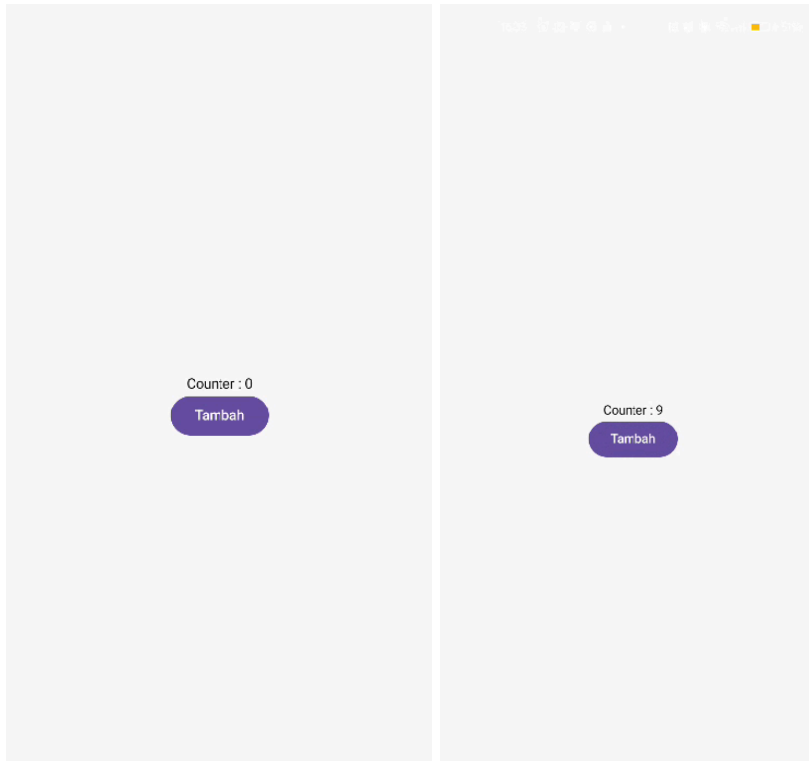
Imperatif berarti memerintah komponen tampilan secara langsung ("cari View ini, ubah propertinya"), sedangkan deklaratif berarti mendeskripsikan tampilan berdasarkan state saat ini dan membiarkan sistem (Compose) yang memperbarui UI otomatis lewat recomposition ketika state berubah. Imperatif fokus pada "bagaimana caranya", deklaratif fokus pada "apa yang seharusnya ditampilkan".

Event Handling State

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

Jawab:

Sebelum & sesudah pencet tombol "Tambah":



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Jawab:

```

----- PROCESS STARTED (16641) for package com.example.pertemuan33b -----
2026-09-16 16:32:37.050 3363-3363 SystemUi--Notification com.android.systemui
2026-09-16 16:32:37.055 3363-3363 SystemUi--Notification com.android.systemui
2026-09-16 16:32:37.183 1969-1969 OplusWifiSarManager system_server
2026-09-16 16:33:32.312 16641-16641 TEST com.example.pertemuan33b
2026-09-16 16:33:33.491 16641-16641 TEST com.example.pertemuan33b
2026-09-16 16:33:33.821 16641-16641 TEST com.example.pertemuan33b
2026-09-16 16:33:33.967 16641-16641 TEST com.example.pertemuan33b
2026-09-16 16:33:34.170 16641-16641 TEST com.example.pertemuan33b
2026-09-16 16:33:34.337 16641-16641 TEST com.example.pertemuan33b
2026-09-16 16:33:34.543 16641-16641 TEST com.example.pertemuan33b
2026-09-16 16:33:34.753 16641-16641 TEST com.example.pertemuan33b
2026-09-16 16:33:34.960 16641-16641 TEST com.example.pertemuan33b
I CommandQueue-->DUMP_STATUS_BAR--> disable pe
I CommandQueue-->DUMP_STATUS_BAR--> disable pe
I wifisar engineerMode state: enable: false, f
D Counter berubah : 1
D Counter berubah : 2
D Counter berubah : 3
D Counter berubah : 4
D Counter berubah : 5
D Counter berubah : 6
D Counter berubah : 7
D Counter berubah : 8
D Counter berubah : 9

```

3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Jawab:

Counter bertambah satu setiap di klik

4. Apa yang menyebabkan hal tersebut terjadi?

Jawab:

Penyebabnya karena count dideklarasikan sebagai var count by remember { mutableStateOf(0) }, bukan variabel biasa. mutableStateOf(0) membungkus nilai count menjadi objek State yang diamati oleh Compose, dan remember membuat nilainya tetap tersimpan (tidak reset) selama composables masih ada di layar. Ketika tombol "Tambah"

diklik, baris `count++` mengubah nilai state tersebut, dan karena Compose mengamati `count`, perubahan ini otomatis memicu recomposition, yaitu Compose menjalankan ulang bagian `Text("Counter : $count")` dengan nilai terbaru. Itulah sebabnya setiap kali tombol diklik, angka di layar langsung bertambah dan terus mengikuti nilai `count` yang baru.

- Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable `EventStateTest`.
- Ubah kode program pada bagian fungsi Composable `EventStateTest` dari yang sebelumnya:

```
var count = 0
```

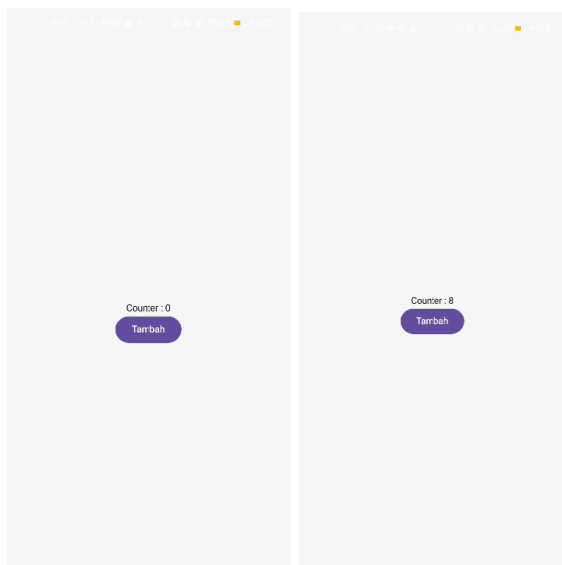
menjadi:

```
var count by remember { mutableStateOf(0) }
```

- Berikut kode lengkap `EventStateTest`:
- Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan `TEST`. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

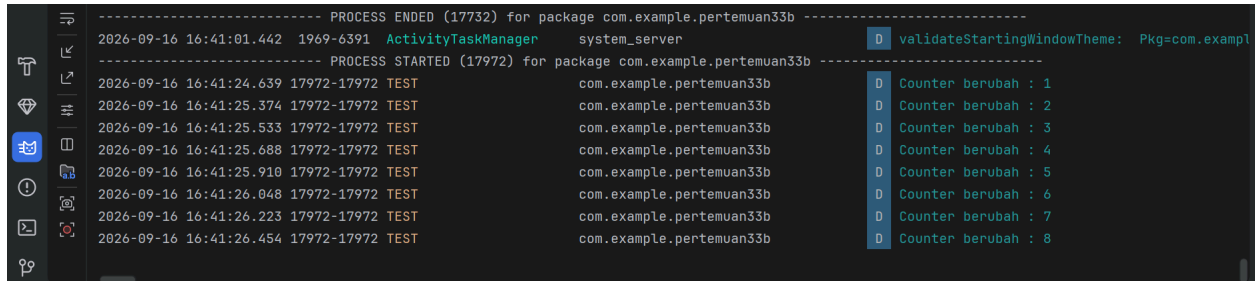
Jawab:

Sebelum & sesudah pencet tombol "Tambah":



- Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Jawab:



```
----- PROCESS ENDED (17732) for package com.example.pertemuan33b -----
2026-09-16 16:41:01.442 1969-6391 ActivityTaskManager system_server D validateStartingWindowTheme: Pkg=com.examp
----- PROCESS STARTED (17972) for package com.example.pertemuan33b -----
2026-09-16 16:41:24.639 17972-17972 TEST com.example.pertemuan33b D Counter berubah : 1
2026-09-16 16:41:25.374 17972-17972 TEST com.example.pertemuan33b D Counter berubah : 2
2026-09-16 16:41:25.533 17972-17972 TEST com.example.pertemuan33b D Counter berubah : 3
2026-09-16 16:41:25.688 17972-17972 TEST com.example.pertemuan33b D Counter berubah : 4
2026-09-16 16:41:25.910 17972-17972 TEST com.example.pertemuan33b D Counter berubah : 5
2026-09-16 16:41:26.048 17972-17972 TEST com.example.pertemuan33b D Counter berubah : 6
2026-09-16 16:41:26.223 17972-17972 TEST com.example.pertemuan33b D Counter berubah : 7
2026-09-16 16:41:26.454 17972-17972 TEST com.example.pertemuan33b D Counter berubah : 8
```

Log dengan tag "TEST" pada package com.example.pertemuan33b itu muncul dari Log.d("TEST", "Counter berubah : \$count") di dalam onClick tombol "Tambah", dan setiap kali tombol diklik muncul satu baris baru dengan nilai yang terus naik ("Counter berubah : 1" sampai "5" sesuai 5 kali klik di screenshot), membuktikan count++ benar-benar berjalan tiap event terpicu; karena count sudah dijadikan state (by remember { mutableStateOf(0) }), selain log bertambah, tampilan angka di layar juga ikut ter-update

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Jawab:

Angka counter selalu bertambah 1

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

Jawab:

Karena count di sini memakai var count by remember { mutableStateOf(0) }, setiap count++ di onClick memicu recomposition sehingga Text("Counter : \$count") ikut ter-update dan angka di layar terlihat bertambah tiap tombol diklik; ini berbeda dengan versi var count = 0 biasa, di mana count hanyalah variabel lokal yang tidak diamati Compose, sehingga meski nilainya bertambah di memori (terlihat dari log yang terus naik), UI tidak pernah digambar ulang dan angka di layar tetap diam. Sementara itu, text di kode ini cuma dideklarasikan tapi tidak aktif karena OutlinedTextField-nya masih di-comment, jadi tidak ada input maupun event onValueChange yang berjalan, satu-satunya event yang benar-benar terjadi hanyalah onClick pada tombol "Tambah".

12. Apa yang dimaksud recomposition pada Compose?

Jawab:

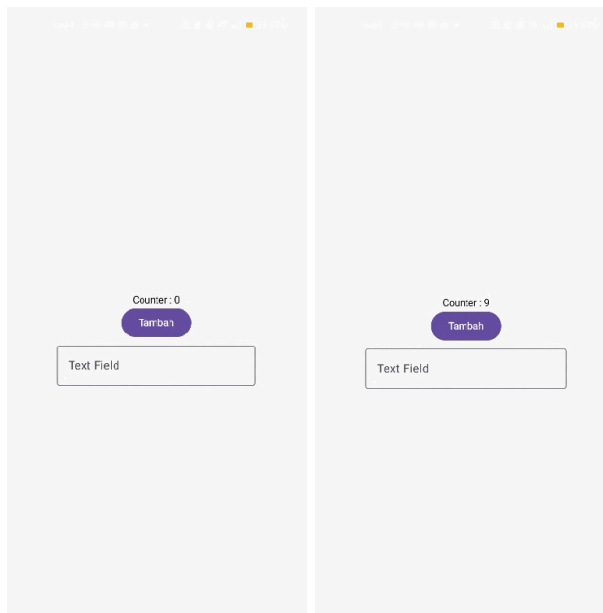
Recomposition adalah proses Compose menjalankan ulang (memanggil ulang) fungsi composible, atau bagian UI yang bergantung pada suatu state, ketika state tersebut berubah, agar tampilan yang ditampilkan sesuai dengan nilai state terbaru. Compose tidak menggambar ulang seluruh UI dari awal, melainkan cukup pintar untuk hanya

menggambar ulang bagian yang benar-benar terpengaruh oleh perubahan state tersebut, sehingga prosesnya efisien.

13. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable EventStateTest. Kemudian di dalam Column panggil composable OutlinedTextField di bawah Button Tambah. Penambahan kode program membuat kode pada fungsi Composable EventStateTest menjadi:
14. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

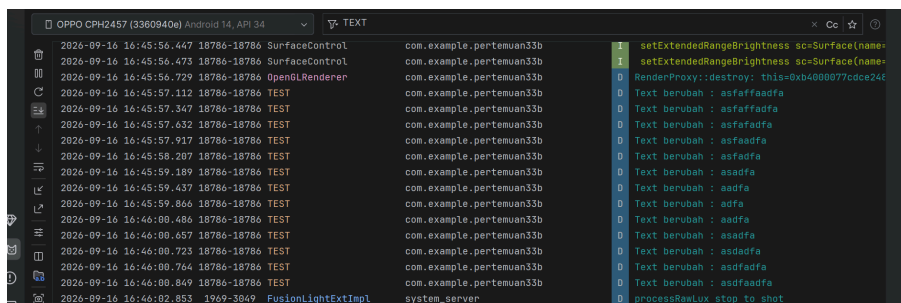
Jawab:

Sebelum & sesudah pencet tombol “Tambah”:



15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Jawab:



Adanya penjelasan bahwa text berubah

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Jawab:

Input field nya menjadi berwarna (aktif) untuk stroke (garisnya) dan apa yang diketik muncul di layar.

17. Apa yang menyebabkan hal tersebut terjadi?

Jawab:

bawaan widget input Android sendiri (EditText/OutlinedTextField): stroke berubah warna saat difokus karena widget itu punya listener fokus internal yang otomatis mengganti warna border dan menggambar ulang dirinya sendiri saat disentuh, sedangkan teks yang diketik langsung muncul karena widget itu memiliki buffer teks (Editable) yang otomatis terhubung ke keyboard dan memperbarui tampilannya sendiri setiap ada input, semuanya sudah ditangani internal oleh class widget tanpa perlu kode manual dari developer seperti `tvMessage.text = ...` pada TextView biasa.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Jawab:

Pada program tersebut ada dua state: count (var count by remember { mutableStateOf(0) }, bertipe Int, nilai awal 0) yang aktif dipakai untuk ditampilkan di Text("Counter : \$count") dan diubah lewat count++ saat tombol "Tambah" diklik, serta text (var text by remember { mutableStateOf("") }, bertipe String, nilai awal kosong) yang sudah dideklarasikan tapi belum aktif digunakan karena OutlinedTextField-nya masih di-comment.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

Jawab:

Pada program tersebut hanya ada satu event yang aktif, yaitu onClick pada tombol "Tambah", yang menjalankan count++ dan Log.d("TEST", "Counter berubah : \$count") setiap kali tombol diklik. Event onChange yang seharusnya ada pada OutlinedTextField belum berjalan karena komponennya masih di-comment sehingga belum ada event terkait input teks pada program ini.

20. Kapan event - event tersebut dijalankan?

Jawab:

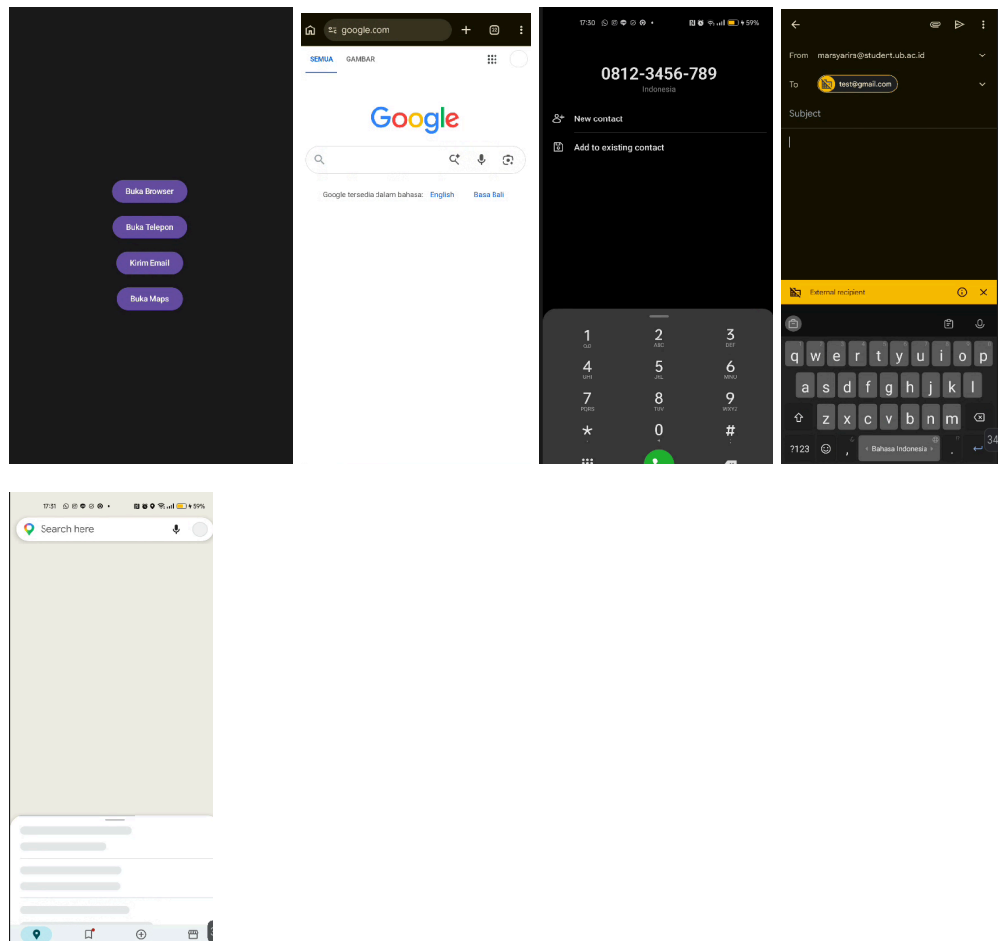
Event onClick tersebut dijalankan setiap kali pengguna menekan/mengklik tombol "Tambah" di layar; setiap satu kali klik akan langsung menjalankan count++ dan mencatat log "Counter berubah : ..." saat itu juga. Karena OutlinedTextField masih di-comment, event onChange belum pernah dijalankan sama sekali, sehingga saat

ini satu-satunya event yang aktif dan bisa terpicu hanyalah onClick pada tombol tersebut.

Implisit Intent

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:
 - Klik Buka Browser
 - Klik Buka Telepon
 - Klik Kirim Email
 - Klik Buka Maps
 - Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya

Jawab:



2. Pada kode yang digunakan, terdapat beberapa action seperti:
 - Intent.ACTION_VIEW
 - Intent.ACTION_DIAL
 - Intent.ACTION_SENDTO

3. Apa perbedaan fungsi dari masing-masing action intent tersebut?

Jawab:

Intent.ACTION_VIEW digunakan untuk menampilkan/membuka data tertentu kepada pengguna dengan aplikasi yang paling sesuai, sifatnya umum (bisa untuk membuka halaman web, lokasi di peta, gambar, dan lain-lain, tergantung jenis data yang diberikan). Intent.ACTION_DIAL digunakan khusus untuk membuka aplikasi telepon dan menampilkan layar panggilan dengan nomor yang sudah terisi otomatis, tapi tidak langsung melakukan panggilan (pengguna masih perlu menekan tombol panggil secara manual). Intent.ACTION_SENDTO digunakan untuk mengirim pesan/data ke tujuan tertentu tanpa lampiran file, seperti pada contoh ini dipakai untuk membuka aplikasi email dengan alamat tujuan yang sudah terisi otomatis.

4. Pada kode yang digunakan, terdapat beberapa data seperti:

- data = Uri.parse("mailto:test@gmail.com")
- data = Uri.parse("https://www.google.com")
- data = Uri.parse("geo:-7.250445,112.768845")

5. Apa fungsi data pada intent ?

Jawab:

data pada intent berfungsi untuk menyertakan informasi spesifik (dalam bentuk Uri) tentang apa yang ingin dituju atau diproses oleh action yang diminta. Data inilah yang memberi tahu sistem Android detail konkret dari aksi tersebut, misalnya alamat website yang ingin dibuka (https://www.google.com), nomor telepon yang ingin dihubungi (tel:...), alamat email tujuan (mailto:...), atau koordinat lokasi yang ingin ditampilkan di peta (geo:...). Tanpa data, sistem tidak tahu target atau isi spesifik dari action yang dijalankan.

6. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Jawab:

Action dan Data saling melengkapi untuk menentukan aplikasi mana yang cocok menangani intent tersebut. Action menjelaskan "apa yang ingin dilakukan" (misalnya melihat/ACTION_VIEW, memanggil/ACTION_DIAL, mengirim/ACTION_SENDTO), sedangkan Data menjelaskan "jenis dan isi target"-nya lewat skema URI (misalnya https://, tel:, mailto:, geo:). Sistem Android mencocokkan kombinasi Action + skema Data ini dengan daftar aplikasi yang terdaftar (lewat intent filter) untuk menangani kombinasi tersebut. Kalau ada lebih dari satu aplikasi yang cocok, sistem akan menampilkan pilihan aplikasi (chooser) kepada pengguna; kalau hanya ada satu, aplikasi tersebut langsung terbuka. Jadi, Action menentukan jenis aksinya, sedangkan Data (khususnya skema URI-nya) menentukan kategori aplikasi apa saja yang relevan untuk ditawarkan sistem.